A Model for Profile Management Applied to Ubiquitous Learning Environments

André Wagner[a], Jorge Luis Victória Barbosa[a], Débora Nice Ferrari Barbosa[b]

[a] *University of Vale do Rio dos Sinos (Unisinos)*
Applied Computing Graduate Program (PIPCA)
Av. Unisinos, 950, São Leopoldo, Brazil
[b] *Feevale University*
Inclusion and Diversity Post-graduation Program
RS-239, 2755, Novo Hamburgo, Brazil

Abstract

Ubiquitous systems have the challenge of implicitly collect relevant information about entities, and use this information to understand and predict their behavior. This allows the applications to adapt themselves to the entities, thus avoiding to overflow them with inquires and information. The analysis of trails, the context-aware history of actions, can further improve the relevance of information. This paper proposes a model that allows applications to register entities' actions in trails and infer profile information from these trails, using semantic interoperability and thus allowing different applications to share information and infer a unified profile. An application was developed and integrated with two different softwares in a context of ubiquitous education, where the student profiles were dynamically updated, allowing them to better adapt to the environment. The contributions of this model are the use of trails for extracting profiles and the capability of managing dynamic inference rules for profile generation.

Keywords:
Profile Management, Ubiquitous Computing, User Modeling, Trails, Profile Inference, Entity-Adapted Interaction

Email addresses: `andre.nho@gmail.com` (André Wagner), `jbarbosa@unisinos.br` (Jorge Luis Victória Barbosa), `deboranice@feevale.br` (Débora Nice Ferrari Barbosa)

1. Introduction

Nowadays, most users interact with a wide variety of computing systems. The current tendency is the increased interconnectedness between these systems, since the internet is becoming every day more present and available. It is predicted the full integration between these systems in a federation of computing systems. This federation is referred to as Smart Environment (Heckmann, 2005; Augusto et al., 2010).

Weiser (1991) discussed a vision for the concept of smart environments. He presented a scenario in which the computer would become invisible and indistinguishable from its environment. In this scenario, interaction between humans and computers would not occur as usual – where the humans devote their full attention to the computer –, but would consist of small and quick interactions, usually in different devices and locations (Weiser and Brown, 1996). According to Weiser (1991), "the most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it." He asserts that the future of computing lies in integrating the computer to the real world, not the opposite, and he calls this new paradigm Ubiquitous Computing.

However, the availability of computing resources at any time and everywhere poses the risk of information overload on the user. To solve this problem, the concept of "calm technology" was created (Weiser and Brown, 1996). This means the transfer of information between the computer and the users must happen in the "periphery" of the users' attention, moving to the "center" of their attention in moments of urgency. A car's engine can be taken as an example. When a person drives a car, the attention is focused on the road, and the driver almost doesn't notice the engine sound. However, if the engine emits a strange sound (that might indicate some kind of problem), the driver's attention turns to the engine.

If a system is to be minimally intrusive, it must understand the situation of the user and his surroundings, and adapt its own behavior according to this information. This information is called Context, and systems capable of this kind of adaptation are called Context-aware (Satyanarayanan, 2001; Dey, 2001).

Research in this area shows that is not only useful to know the context of the user when a service is being offered, but that the history of visited contexts can be a valuable information to help the application to adapt itself to the user. This history of visited contexts is called Trail (Driver and Clarke,

2008). Although the trail is usually tied to the location of a user, the concept of a trail is broader and includes activities performed, applications used, content accessed – among others – inside a specific context and timeframe (Silva et al., 2010).

For an application to adapt itself effectively to a user, it is not only important to know the history of contexts visited by a user, but to understand who the user is – that is, what are his characteristics. This kind of application organize its knowledge about a user in User Profiles. The profiles are stored in a format called User Model (Fischer, 2001; Viviani et al., 2010).

Personalization means the individual adaptation of products, services and information. Personalization technology usually involves programs that learn patterns, habits and preferences of a user. The main goal of modern personalization systems is to offer the users what they want, without demanding them to inform it explicitly (Mulvenna et al., 2000). It is clear that this purpose meets the ubiquitous computing objective that is, the development of a technology that "disappears".

In this paper, the concept of *entity* is used instead of the classic concept of *user*. While usually the "user" means the person who is using the application, an entity refers to any object (physical or not) able to control an application. Thus, an entity can be a person, a moving object (like a car) or property (like a room), or even a computer system.

A relevant aspect for environments with multiple independent systems is the interoperability. Interoperability is the ability of two or more systems or components to exchange information and use the information that has been exchanged (Geraci, 1991). In the context of customization, interoperability refers to the ability to access and interpret information derived from multiple heterogeneous sources, and integrate it into a common user model. Interoperability can be (i) syntactic, where the differences between the models are resolved at application level, or (ii) semantic, where the differences between the models are resolved at the level of knowledge (Carmagnola and Cena, 2009).

Research indicates that it is desirable that future systems become increasingly invisible to their users (Weiser and Brown, 1996), and that a system can only become invisible if it is proactive, and can only be proactive by knowing the user (Satyanarayanan, 2001). For a system to know a user, it must know the user history (Silva et al., 2010), understand the context in which he is inserted (Dey, 2001), as well as data collected concerning him in the same context (or in a similar context), and be able to process this set of

information into a user profile (Fischer, 2001; Viviani et al., 2010).

In this setting we are proposing eProfile. eProfile is a ubiquitous profile manager of generic domain oriented towards distributed environments, with inter-system interoperability, which infers profile data through analysis of an entity trails (provided by a manager tracks), following rules defined by the entities, and provides data inferred to be used by applications.

Ubiquitous computing has been used successfully in several different environments and contexts, such as commerce (Franco et al., 2011), logistics (Oliveira et al., 2013), accessibility (Tavares et al., 2012) and entertainment (Segatto et al., 2008). One such environment is ubiquitous learning, or U-Learning. Ubiquitous learning can be understood as a set of educational processes that occur anywhere, at any time and with any device, continuously, contextualized and integrated into the daily lives of apprentices (Barbosa et al., 2011). It is important to notice that U-Learning is more than simply "learning with u-computing technology" (Hwang et al., 2007), but is composed by a mobile environment that takes into consideration the learner's individual information of various sources in context, and adapts the instructional content to meet the functions of different mobile devices (Hwang et al., 2008).

eProfile's main characteristics are the use of trails for extracting profiles, the capability of generating dynamic profiles, the capability of managing dynamic inference rules and models, and the semantic operability of the model. These characteristics are used to maintain the student dynamic profiles updated, allowing a better adaptation to the ubiquitous learning environment.

This paper is organized as follows. Section 2 is dedicated to related works. Section 3 proposes the eProfile model. Section 4 discusses implementation aspects. Section 5 presents an eProfile integration with two existing applications in a scenario of ubiquitous learning. Finally, Section 6 presents the concluding remarks.

2. Related Works

The UUCM model, presented by (Niederée et al., 2004), works in a generic domain. It does this by creating a basic ontology that can be extended by applications. Unlike many other proposals, UUCM stores the profiles on the client rather than the server. This brings advantages and disadvantages. Among the advantages there can be mentioned the privacy and disconnected access. However, this approach limits the interoperability between systems,

since each client gets a version of the client's profile, restricting the use of a unified profile.

Kim and Lee (2008) presented a framework for profile management based on Web Services. The model provides a set of services to gather information necessary to create a profile: configuration, creation, exchange, processing and service. The model also applies dynamic configuration techniques for efficient profile management.

PeLeP (Levis et al., 2008) supports automatic improvement of a learner's profile in education ubiquitous environments, using the history of apprentices to update the profiles through inference. However, since PeLeP's inference is made on a single specific domain (education), the inference rules are defined by the model and not the application.

Hong et al. (2009) presented a model for a context-aware system for a proactive service based on context history. In this model, the context history of users is analyzed by an agent-based framework that provides personalized services. The authors implemented a prototype system using the proposed framework that offers services to the users according the context being used in their PDA.

The model presented by Chellouche et al. (2010), presents a unified profile which can be used for various applications. The model assumes syntactic interoperability, which means that all applications need to record the events using the same format, which can cause the application to not be able to register the event with the richness of detail required. Moreover, the model presented by Chellouche et al. (2010) is limited to a specific domain (multimedia services).

The model presented by Razmerita (2011) aims to link traditional KM (Knowledge Management) with personal KM. This is done through OntobUMf model, that manages user information. The architecture has three layers: front end, services and data. The model works in a generic domain and has three ontologies (user, domain and log), from which are fetched information that categorize the user in various ways, according to the level of activity, type of activity and level of knowledge sharing.

Wu et al. (2012) presented a ubiquitous and context-aware learning strategy using a cognitive apprenticeship approach applied to the standard operating process of a physical assessment nursing course that includes collecting patient' life signs and other information, where the students used a RFID-based mobile system that helped them on their learning. Experiment showed that the students' skill level improved more using a ubiquitous learning sys-

tem than the traditional approach.

In the paper presented by Tao et al. (2011), it is proposed an ontology model for knowledge representation to gather personalized information. The model builds personalized user ontologies by extracting global information from LCSH[1] system and by discovering information from local user repositories. The local information system is fed manually by the user. When information is needed, it processes both ontologies to generate the requested data.

Table 1 shows a comparison between related works. The following factors are taken into consideration:

- Interoperability: it considers if the model presents interoperability – if yes, whether it is syntactic or semantic – according to the definition in Section 1;
- Dynamicity: it identifies which aspects of the profile management are dynamic. "Profiles" means that the model allows the profiles to be updated according to external events, without user interference, "Models" means that the entity or user models can be updated during execution, and "Rules" means that the rules for profile generation cab be changed on runtime;
- Storage: it defines where the user profiles are stored;
- Context awareness: considers if the user context is taken into consideration when forming the profile;
- Uses trails: it defines if the trails are used when building the profile. In the table, "Partial" means the model uses some type of historical information to form the profile, but doesn't record the history of visited contexts and the actions done within these contexts;
- Domain: it identifies if the profile is oriented to a specific domain, or if it allows different domains to be used.

The main characteristics of eProfile's model are:

[1]Library of Congress Subject Headings

Paper	Interoperability	Dynamicity	Storage	Context awareness	Uses trails	Domain
Kim and Lee (2008)	Syntactic	Profiles	Server	No	No	Generic
Chellouche et al. (2010)	Syntactic	Profiles	Server	Yes	No	Multimedia Services
Niederée et al. (2004)	Syntactic	Profiles	Client	Yes	No	Generic
Levis et al. (2008)	No	Profiles	Server	Yes	Partial	Education
Razmerita (2011)	Syntactic	Profiles	Server	Yes	Partial	Generic
Tao et al. (2011)	Semantic	Profiles	Server	Yes	Partial	Generic

Table 1: Comparison between related works and eProfile

1. The use of trails for extracting profiles. By looking at the entity history, it is possible not only to consider its current situation, but also to examine the entity's past, obtaining a more complete set of information. Furthermore, this allows the entity profile to be generated automatically, instead of requiring the entity or application to fill out the data manually;
2. The capability of generating dynamic profiles. The continuous creation of trail events allows the profile to be constantly updated, thus allowing the automatic adaptation of the profile to the entities' actions;
3. The capability of managing dynamic inference rules and models. The applications can change the inference rules and entity models at any moment, which give the developers a opportunity to build applications that can adapt themselves to their users;
4. The semantic operability of the model. The ability to use information generated by various applications to build an entity profile allows the application to create more comprehensive profile generation rules, thus enriching the profiles information.

3. eProfile Model

This section describes the eProfile model. Section 3.1 presents a general vision of the model. Section 3.2 explains the model's architecture. Section 3.3 demonstrates the ontologies used by eProfile, and Section 3.4 explains how the profiles are inferred.

3.1. General View

Figure 1 shows the overview of eProfile model, where each circle represents a component, and the arrows indicate the flow of data.

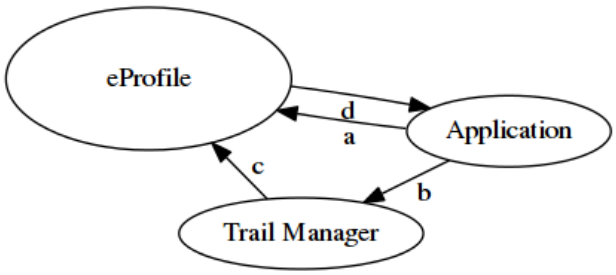

Figure 1: Overview of eProfile

The **application** is any piece of software that is being used directly by an entity. For an application to be considered suitable to use this model, it needs to record its information on a trail manager supported by eProfile. The application must also implement a communication protocol to exchange messages with eProfile. If these prerequisites are followed, the application can be of any type and run in any type of device that has communication with the Internet.

The second component depicted is a **trail manager**. A trail manager is capable of storing events held by an entity, putting them in chronological order and registering information relevant to the event (e.g., the context in which the event was held). The eProfile supports any trail manager whose communication protocol is implemented in eProfile.

The third component of Figure 1 represents the **eProfile**, where the arrows indicate the flow of data. The communication starts when the application registers itself on eProfile (arrow "a"), sending (i) the entity model used by the application and (ii) the rules for construction of this model. From this moment on, the application starts recording the events on the trail manager (arrow "b"). The eProfile periodically retrieves this information from the trail manager and builds or reconfigures the profile entity (arrow "c"). When the application needs an updated profile, it retrieves this profile from eProfile (arrow "d").

Since eProfile supports multiple applications, it is possible for different applications to use different trail managers. Therefore, eProfile also supports communication with a number of trail managers, as exemplified in Figure 2.

3.2. Architecture

Figure 3 shows the architecture of eProfile. The components "Application" and "Trail Manager" are the same as shown in Figure 1, while the

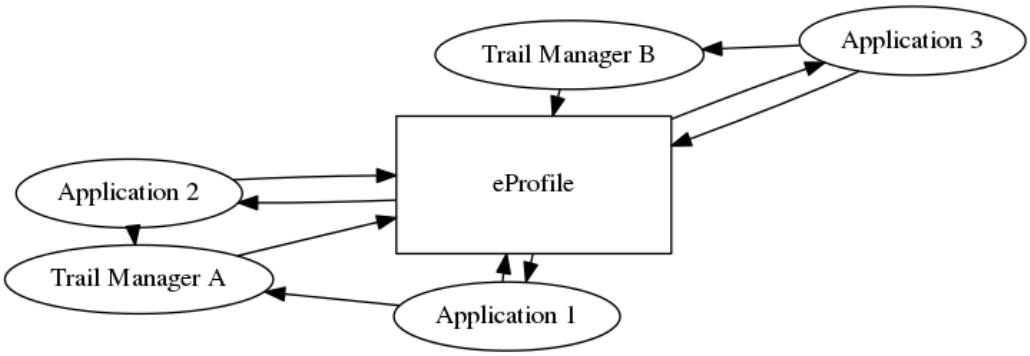

Figure 2: eProfile connected to multiple trail managers

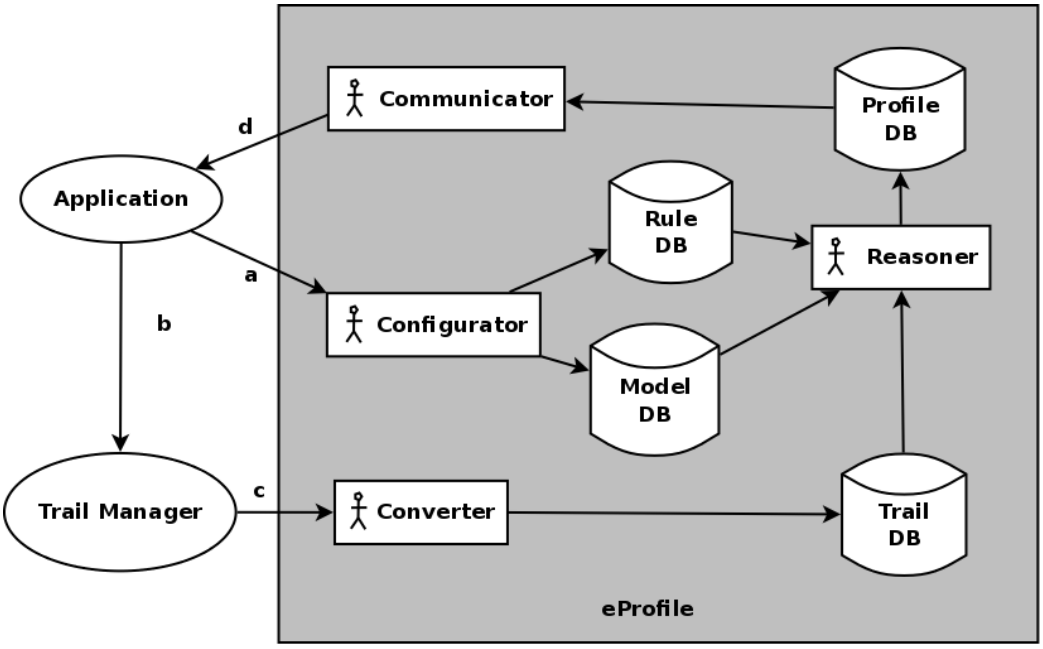

Figure 3: eProfile architecture

"eProfile" was expanded to show its inner workings. The letters used in this figure represent the same ones used in Figure 1. The eProfile is composed by four autonomous agents and four databases. The usage of agents simplified the processing of ontology data and the inference execution.

Each application builds its own entity model. The eProfile needs three pieces of information to be able to process the profiles: (i) the entity trail, (ii) the entity model used by the application and (iii) the inference rules for the profile. This can be seen in Figure 3, where the agent **Reasoner** receives data from these three databases. The trail of the entity is loaded by eProfile from the trail manager, while the other two information are sent directly from the application to eProfile. Thus, the inference rules defined by the application are used to infer the profile of the entity.

The arrows in Figure 3 indicate the direction of the data flow, where it can be noted two entry points and one exit point.

The first entry point (arrow "a") is the information sent from the application to eProfile or, more specifically, to the **Configurator** agent. The following information must be sent by the application:

- The entity model used by the application, which is the format of the entity profile. Once the entity trail is processed, the inferred data is stored and made available for application using this format;

- The rules used to infer the profile. These rules will be executed at regular intervals in order to update the entity profile. The Configurator receives the rules, validates them and inserts them in **rule database**. Along with the rules, is specified the criticality of information. The model uses this information to determine how often the profiles will be inferred.

The second entry point (arrow "c") allows receiving the entity trails, which are fetched by eProfile from the trail manager. Since eProfile can connect to different trail managers, (see Figure 2), and each trail manager might use one specific format, a conversion between the original format and the format used by eProfile is needed for the trails to be stored in a standardized manner. This work is performed by an agent named **Converter**, which is able to perform the conversion to the eProfile internal trail format.

The entity trail is then stored in a trail ontology called **eOntoTrail**. This ontology contains the basic classes supporting the construction of a trail, and

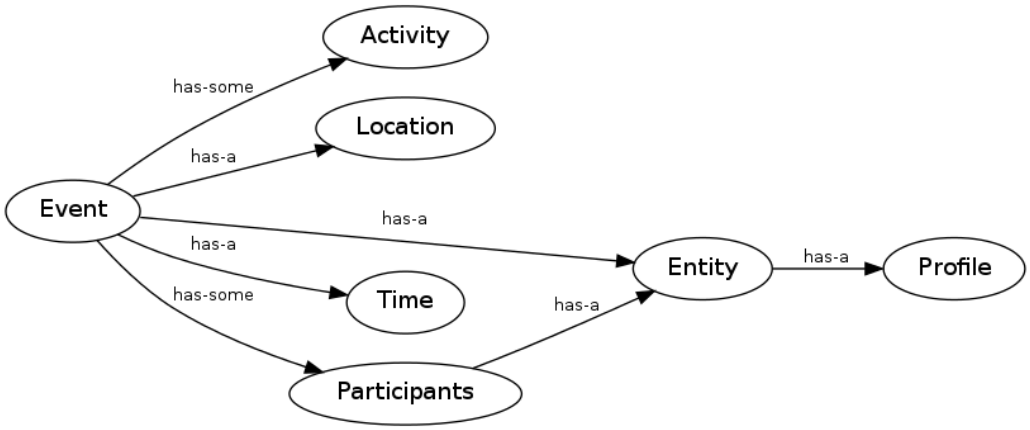

Figure 4: eProfile Trail Ontology (eOntoTrail)

they can be extended according to the specification of the trail manager that is being used.

Once in possession of the trails, the model and rules, the **Reasoner** agent infers from the trail, generating a entity profile that consists of an extension of a basic profile ontology named **eOntoProfile**. This ontology, just like the trail ontology, contains basic classes from which applications can specialize, using them to store their own information. The application can then connect to eProfile and get its entity profile, whose data are sent by the **Communicator** agent (arrow "d").

3.3. Ontologies

As mentioned in Section 3.2, eProfile uses two ontologies: a trail ontology and a profile ontology. These are both basic ontologies, extended by subclasses according to the trail format sent by the trail manager (for the trail ontology) or the needs of the application (for the profile ontology).

The trail ontology data (eOntoTrail), shown in Figure 4, is fed by different trail managers. The Converter agent is responsible for receiving data in the trail manager format and converting it to the eOntoTrail. This way, the reasoner has a single ontology on which to reason.

Context-aware applications are interested in knowing **who**, **where**, **when** and **what** (that is, what the user is doing), so they can discover **why** the situation is occurring (Abowd et al., 1999). Thus, a trail is represented by a set of events characterized by the following main classes: (i) *Entity*, the

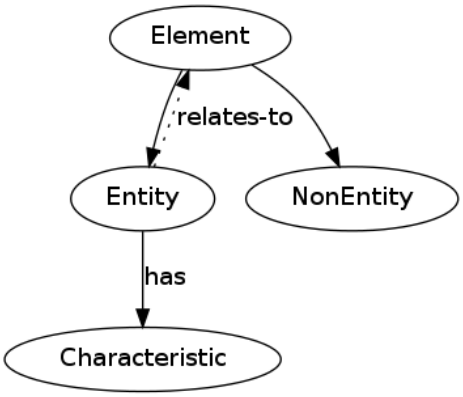

Figure 5: eProfile Profile Ontology (eOntoProfile)

identity of the entity that is performing the action, which can optionally be linked to a profile, (ii) *Location*, where the action is taking place, (iii) *Time*, the date and time the event occurred, (iv) *Participants*, the entities linked to this action, and (v) *Activity*, the action that the entity is performing (or trying to perform). This last class is the most comprehensive of the ontology, and will possibly be the most specialized by the trail managers.

The profile ontology (eOntoProfile), shown in Figure 5, is the ontology used in the profile information inferred from the trail, according to the entity model defined by the application. For the ontology to be feasible within the proposed model, it must be a generic ontology that allows its classes to be extended by applications. In the figure, a relationship between classes is shown by a dotted line, and a specialization of a class is shown by a continuous line. The eOntoProfile is a basic ontology, extended by the applications to assemble their own entity models.

The basic class of the ontology is the *Element* class. This class represents anything in the real world. It is extended by two classes: (i) *Entity*, which represents an entity, that is, an element that has a profile associated with it, and (ii) *NonEntity*, which represents an element than doesn't have a profile associated with it. The class *Characteristic* represents an aspect of the entity, and is the class that will be extended by applications to refer to any information related to that entity. These characteristics can be simple labels (such as name of a person, or mileage of a car) or more complex information (such as academic knowledge or latest routes traveled).

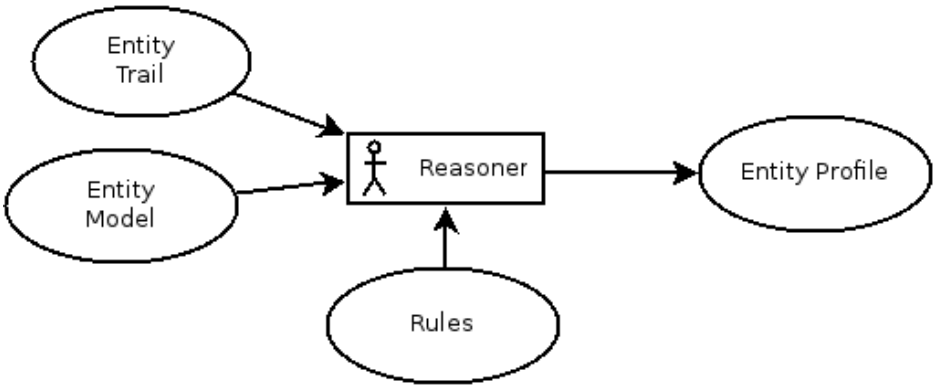

Figure 6: Data flow for the entity profile inference

3.4. Rule Execution

The execution of rules is key to eProfile's operation. An inference rule is an instruction issued by an application, that runs on an entity trail, resulting in an entity profile according to an entity model previously registered by the application.

For a rule to be executed, the application must have previously registered within eProfile: (i) the rule itself, (ii) an entity model, which serves as basis for the entity profile format, and (iii) the trail of the entity on which the inference is performed. From these three pieces of information, the reasoner is able to generate an entity profile, according to Figure 6.

The rules are always defined by the application, and a rule is composed of the following set of information: (i) rule name, (ii) profile field that will be updated, (iii) the rule, written in OWL language (W3C, 2011), (iv) update method (replacement or update), (v) update frequency, and (vi) expiration of the rule.

The update method defines how the value will be changed in every execution of the rule. There are two options: (i) replacement, where the value will be replaced by a new information (for example, in the case of a person who changes its address); and (ii) update, where the value is updated according with a formula, that can take into consideration the current value (before the update). The update option could be used, for example, in a field that stores the interest of a person in a particular subject. The interest doesn't change radically, but is changed smoothly based in the previously registered interest.

The update frequency defines how often the inference process will be

executed, updating the profile. The expiration of the rule defines for how long a rule will be valid before it is deactivated.

The rules and the entity model can be changed at any moment during the execution of eProfile. A change in the rules or in the model will reflect in a change in the generated profile when the rules are executed the next time.

4. Implementation

eProfile prototype was developed in Python (Rossum, 1995). Python is a generic, open source, object-oriented, high-level language, that supports multiple architectures.

The rule, model and profile information are stored in SQLite3 databases (Hwaci, 2013). SQLite3 is an SQL transactional self-contained database, available in the public domain. The SQLite3 was chosen due to its good performance and the fact that it does not require a separate server for deployment. The use of a relational database to store the rule, model and profile databases was made due to performance reasons.

The Reasoner agent uses an external inference engine to implement the rules. The reasoner used is FaCT++, a propositional logic reasoner created as a platform for use of algorithms of semantic *tableaux* and optimization techniques (Tsarkov and Horrocks, 2006). This reasoner was chosen due to its good performance and to support all functions used by eProfile.

The knowledge representation language used for storage and processing of ontologies is OWL(W3C, 2011), chosen for being the most expressive ontology language defined for the Semantic Web (Daconta et al., 2003). Communication between eProfile, the trail managers and applications is done through REST web services (Fielding, 2000).

5. Evaluation Aspects

It is only possible to evaluate the functionalities of a proposed infrastructure by building applications that utilize it and then evaluate them, in order to obtain an indirect evaluation of the model (Edwards et al., 2003). The scientific community has been using scenarios to perform this type of indirect evaluation in context-aware (such as the approach taken by Dey et al. (2001)) and ubiquitous (according to Satyanarayanan (2001)) environments. This work follows the same approach – we created a prototype and a scenario for performing an experiment, in order to evaluate eProfile.

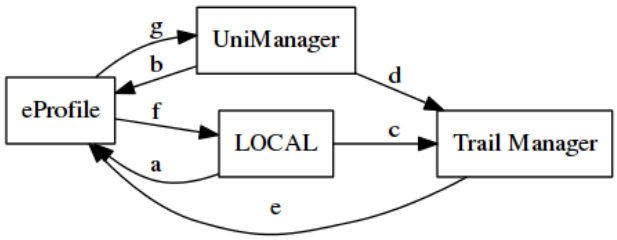

Figure 7: Scenario model

The application presented is based on the generation of simulated trails of undergraduate students within the context of Ubiquitous Learning. To build this application, eProfile was integrated with two softwares called LOCAL and UniManager.

The following sections describe the applications used for the integration (Section 5.1), the eProfile/LOCAL integration (Section 5.2) and eProfile/UniManager integration (Section 5.3), the simulation environment (Section 5.4), trails, rules and generated profiles (Section 5.5) and the simulation parameters and results (Section 5.6).

5.1. Applications

The integration is composed of four applications that interact among themselves, according to Figure 7. LOCAL is a ubiquitous learning system used to find learning opportunities by putting students in touch and recommending learning content (Barbosa et al., 2011). UniManager is an application used by university administration to manage students grades and disciplines.

Both LOCAL and UniManager have profile modules, and the profile information is used for making decisions. Since in both cases the profile module is static and depends on collection of explicit information, eProfile is connected to these softwares, leveraging the dynamic creation of profiles and collecting information implicitly. Furthermore, eProfile enables interoperability between these systems by allowing information from both systems to be used to generate single profiles. The scenario also includes a trail manager, that manages the entities trails. In this scenario, the disciplines and students are represented as entities.

The initial communication occurs between applications and eProfile (arrows “a” and “b”), where applications define their entity models and rules.

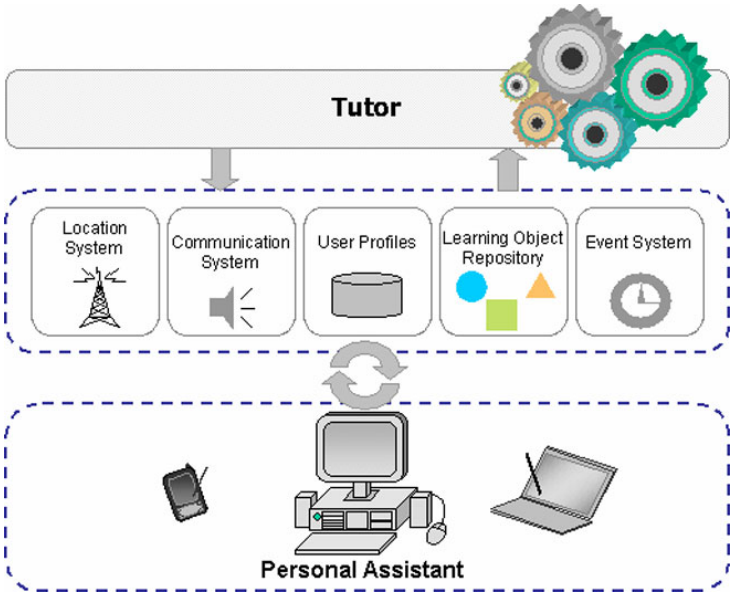

Figure 8: LOCAL model (Barbosa et al., 2011)

This data exchange can occur as often as necessary, because applications can update their rules and their entity models at any time. During the execution of the applications, they record the events in the trail manager (arrows "c" and "d"). This means that applications can run disconnected from eProfile, and connect only when a rule needs to be updated or when profile data must be downloaded. eProfile retrieves trails periodically from the trail manager (arrow "e"). When eProfile is in possession of the trails, rules and entity models, it starts to process this information, inferring new profile data. These profiles are then downloaded by the applications (arrows "f" and "g").

5.2. eProfile/LOCAL Integration

This scenario integrates eProfile to a ubiquitous learning system called LOCAL (Location and Context-Aware Learning). LOCAL uses location information and context as an aid to teaching and learning process. A location system monitors the mobility of apprentices and, based on their physical position, explores educational opportunities (Barbosa et al., 2011).

LOCAL architecture is shown in Figure 8. The gathering of LOCAL's profile system information is explicit. This means that users' (learners) relevant data need to be pre-populated in the system. Furthermore, the LOCAL

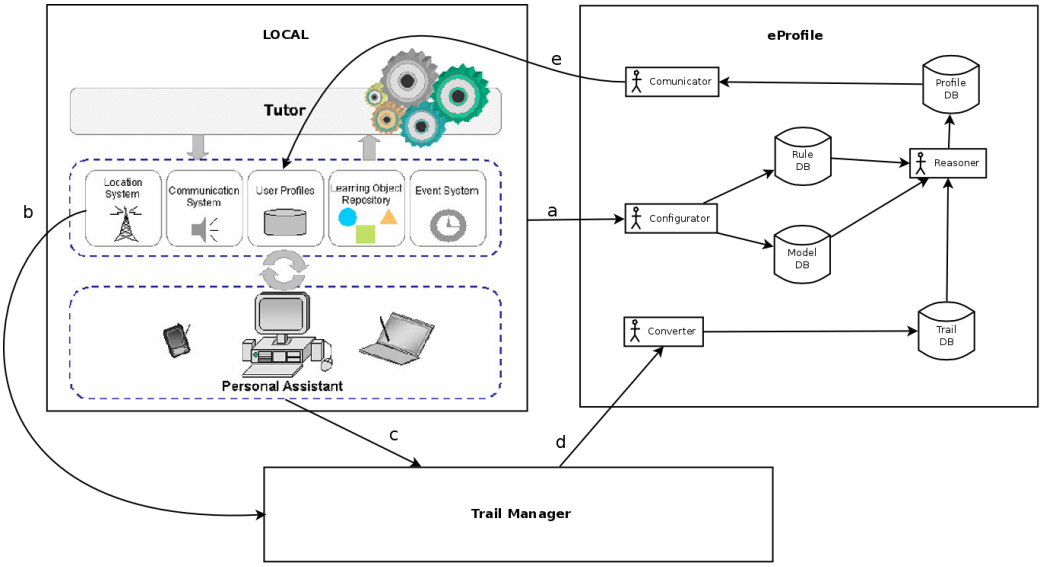

Figure 9: eProfile/LOCAL Integration

profile system is static, which means that data is not automatically updated in response to actions taken by learners or external events. Since eProfile is a dynamic entity model with implicit information collection, integrating these two models result in the enhancement of LOCAL capabilities.

LOCAL and eProfile integrate according to Figure 9. Besides LOCAL and eProfile, Figure 9 presents an application called Trail Manager. The Trail Manager is an external program that stores the trails, processes them and forwards them to eProfile, as described in Section 3. For this scenario, we created a simple trail manager, which just receives, stores and forwards the trails to eProfile upon request.

Integration between eProfile and LOCAL was based on the following changes to LOCAL: (i) we included a connection interface with eProfile, able to generate learners entity models and rules for profile processing, (ii) we included a connection interface with the trail manager, to allow the location events and actions undertaken on the Personal Assistant to be sent to the trail manager, and (iii) we included a communication interface with eProfile, where the LOCAL User Profiles module is fed by profiles generated by eProfile.

As learners use LOCAL, their location changes are identified by the Location System component, and their actions are done on the Personal Assistant.

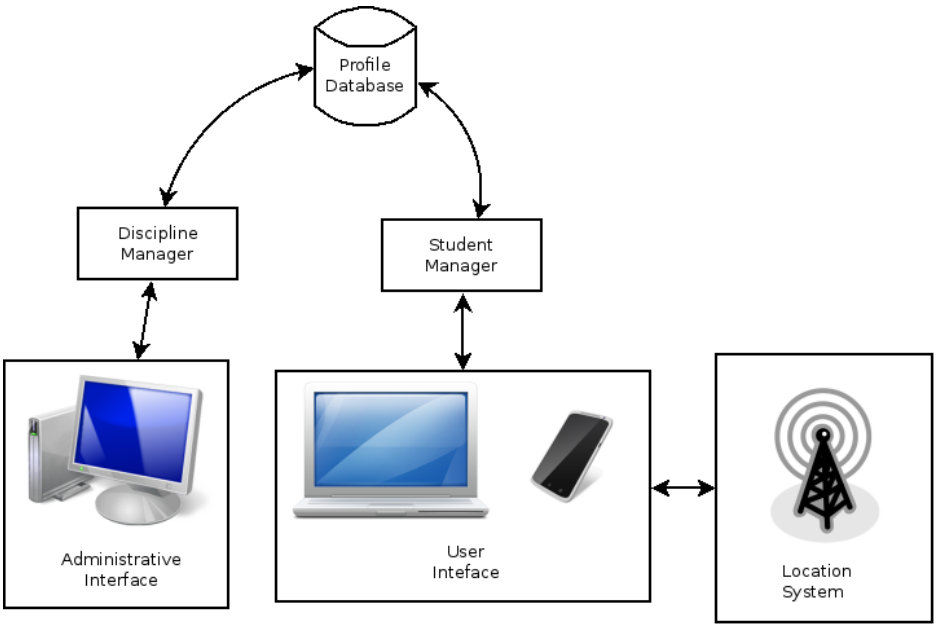

Figure 10: UniManager model

These components record events in the Trail Manager (arrows "b" and "c"), which in turn sends them to the Converter eProfile agent (arrow "d"). Trails sent are stored in the Trail database. These trails are periodically processed according to the models and rules defined initially, and a inference process based on this information generates profiles, which are sent to the LOCAL User Profiles module (arrow "e") by the Communicator agent. These profiles help LOCAL to know its users (learners) in order to put them in contact and distribute content in the most contextualized possible way.

5.3. eProfile/UniManager integration

UniManager is a administrative software used by the university for registration and management of students and disciplines. This application model is shown in Figure 10.

UniManager has two interfaces: administrative and user. The administrative interface is used by the management of the university to enroll disciplines and students. It is also used by students to carry out their registration for the next semester. This interface connects to a discipline manager, which manages the enrollment and prerequisites between disciplines.

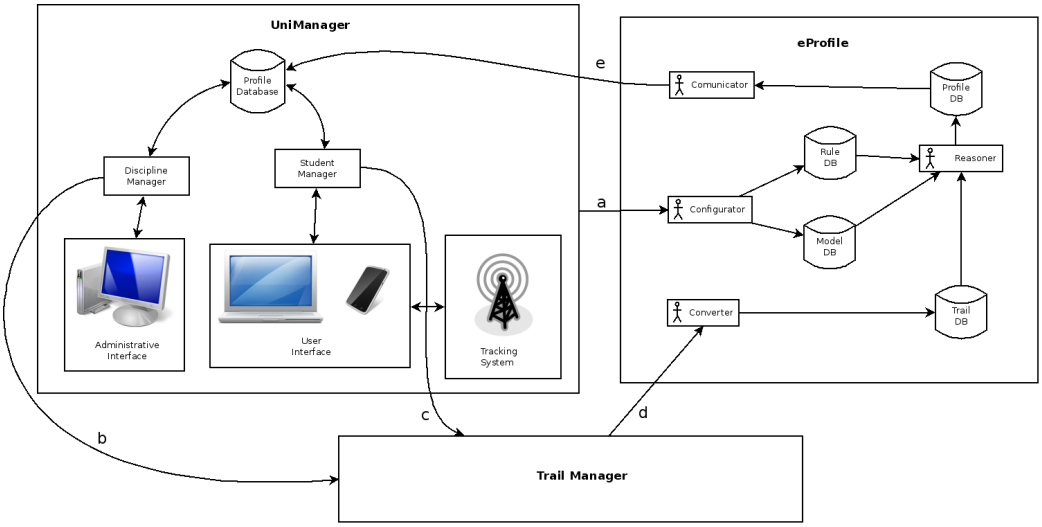

Figure 11: UniManager/eProfile integration

The user interface is used in the students' mobile devices, and is connected to the university location system. Thus, all events recorded in UniManager from this interface contains the location of the student within the university, that is, the campus' region where the student is. The location system integrated to the user interface allows the system to check whether the student is present in class. The student performs the tests and hands out papers using this interface.

UniManager also contains a profile database. On this database are stored: (i) registration information about the disciplines, (ii) presences and absences of students, (iii) tests and papers handed out by the students, and (iv) the evaluation of student work.

The profile database is dynamically updated based on recorded events, but is limited to data recorded in UniManager itself. Integration between eProfile and UniManager leverages the generation of more advanced profiles, using the information inferred by eProfile from the data generated by LOCAL. Using this data, it is possible to register the interests of the student in UniManager's profile database, in order to suggest him optional subjects that might be of his interest. Furthermore, the use of trails allows the profiles to be generated based on historical information.

UniManager integrates to eProfile according to Figure 11. In the first communication between UniManager and eProfile, UniManager sends mes-

sages indicating to eProfile which are the entity models and the rules used to generate the profiles (arrow "a"). This information is received by the Configurator agent, responsible for validating and storing the rules and models in their respective databases. As users make use of the UniManager interfaces, data is processed and stored by the Discipline and Student Manager modules in the trail manager, along with their location (arrows "b" and "c"). The trail manager used by UniManager is the same one used by LOCAL. The trail manager periodically sends the trails generated by UniManager to the Converter agent in eProfile (arrow "d"). This agent converts the trails to the eProfile format and stores them in the trail database. Once in possession of the trails, models and rules, the Reasoner agent processes the information and generates profiles that are sent to UniManager by the Communicator agent (arrow "e").

5.4. Environment

The scenario consists of a group of students that carries out a graduation course in Computer Science. The course consists of a total of 40 disciplines, of which 12 are optional (the student must complete at least 8 out of these 12). All disciplines have other disciplines as prerequisites, except for the disciplines of the beginning of the course.

Students use mobile devices (laptops or smartphones), on which the applications used for administration of their academic life are installed – in particular, UniManager and LOCAL.

UniManager is the system used for discipline administration, and it is described in detail in Section 5.3. It is used by the university administration and the students. The administration uses it to register disciplines, and students, to link students to their disciplines and to issue management reports such as attendance and approval/failing of students. Students use it to hand out their tests and papers, to make their enrollment in the disciplines and to check their grades and absences.

In addition, professors and students also use LOCAL on their mobile devices. LOCAL is described in more detail in Section 5.2 and in (Barbosa et al., 2011). LOCAL links profile information with the user location, facilitating interaction between learners and enabling the delivery of contextualized educational material. Professors use the LOCAL to register material that will be distributed to students. LOCAL communicates with students to distribute educational material previously registered and to put students with similar interests in touch during educational activities. If, during these

activities, the student has not registered any interest that matches those required by the activity, he has the option to make the choice using a user interface to LOCAL called Personal Assistant.

Table 2 exemplifies a two and a half hours class involving 10 students, in which there was a debate about Programming Techniques. After the debate, students undertook a study about the individual topics discussed in the debate. In this example, the presence of students, the handing out of their work and the grade achieved were registered in UniManager. The educational content of the lesson was registered in LOCAL.

In this example, LOCAL registered the personal preferences of two students about programming techniques. If LOCAL and UniManager were being used in isolation, this information would have been relevant only to the immediate lesson assignment. However, the integration of these systems with eProfile allows this information to be used to update the profile of the students, and can be used in the future to facilitate interaction between students and personalize the delivery of educational content (by LOCAL). Furthermore, a single profile can be generated by the trails from both applications, this information can be used to assist the student in the recommendation of optional disciplines (by UniManager).

5.5. Trails, Rules and Profiles

According to what was described in Sections 5.2 and 5.3, LOCAL and UniManager suffered three significant changes: (i) there were included communication interfaces with eProfile, so that applications can provide the entity models and rules, (ii) there were included communication interfaces with the trail manager, so that applications could register the events and (iii) the profile modules of both applications were modified to request profile updates from eProfile.

The scenario consists of two types of entities: students and disciplines. The "student" entities represent each one of the students involved in the course. The "discipline" entity refers to each discipline of the course. A discipline could be represented either as an entity or as a single resource; in this scenario, we decided that it would be represented as an entity, since this allows it to create a trail, from which a profile can be obtained. This way, the profile of the discipline can contain fields that are updated automatically such as, for example, the average grade of the discipline.

Event	Time	Actor	Action
1	-	Professor	Registers educational material that will be used in class into LOCAL, and requests the handing out of a paper through UniManager.
2	7:30	Students	Nine students enter the classroom.
3	7:30	LOCAL	LOCAL identifies the entrance of the students into the classroom.
4	7:30	UniManager	UniManager identifies the entrance of the students into the classroom.
5	7:30	LOCAL	LOCAL sends the class educational material to the students.
6	8:00	Professor	The professor requests the formation of groups for the debate.
7	8:00	LOCAL	LOCAL forms three groups based on the students' preferences: *Distributed Algorithms* (3 students), *Scheduling Algorithms* (3 students) and *Math Algorithms* (1 student). Two students have no registered preference that would allow LOCAL to put them in some group.
8	8:00	LOCAL	LOCAL request the two students to choose a group.
9	8:10	Students	Students indicate their preference through LOCAL Personal Assistant.
10	8:10	LOCAL	LOCAL puts the two students in their chosen groups, one of them in *Scheduling Algorithms* and the other one in *Math Algorithms*.
11	8:10	Students	The debate starts.
12	8:40	Students	One student enters the classroom, late. UniManager and LOCAL register his entrance, but LOCAL doesn't integrate him in the debate, since it's almost over.
13	8:50	Students	The debate ends.
14	8:50	Professor	The professor request the students to write a paper about the debate subject.
15	10:00	Students	The students hand out their papers through UniManager and leave the classroom.
16	10:00	UniManager	UniManager receives the papers and registers the students exit.
17	10:00	LOCAL	LOCAL registers the students exit.
18	-	Professor	The professor, later, evaluates the students papers and registers their grades on UniManager.

Table 2: Example of a class

5.6. Prototype and Simulation

The scenario objective was to evaluate the integration between eProfile, LOCAL and UniManager through a four year undergraduation course, which made a real-life scenario impractical. A simulation was thus created to validate the model. Two programs were developed:

1. A simulator, which simulates students, professors and disciplines using UniManager and LOCAL;
2. A trail manager, which stores the generated trails and makes them available to eProfile.

To simulate the scenario as faithfully as possible, each entity was created with a set of characteristics that distinguish them from one another, and causes them to have different performances and take different decisions when faced with a choice. These characteristics directly influence the quality of student learning, the difficulty of the course and the likelihood of dropout students. Students are generated with varying levels of commitment, intelligence and personal tastes. These characteristics influence student performance, the probability of withdrawal of disciplines and course, and the choices that the student will make.

The scenario simulates a group of 40 students who conduct a course in Computer Science at a university. A simulation starts with the creation of the discipline and student entities. The initial profiles of the disciplines are then generated. A loop then start, and it will only end when all students have completed the course or dropped out. Each iteration of the loop is a semester course and contains the following steps:

1. Students choose the disciplines they will attend and enroll;
2. Each day of the course is simulated, where students attend class and perform tests or deliver papers;
3. The students final grades are calculated, and the students decide if they will continue on the course or drop out.

Each course consists of 14 classes. There are two types of events that can occur in a class: (i) a debate or group work, and (ii) an individual paper or test. A class can have one of these two events, both or neither. Table 2 exemplifies a class in which both types of events occurred.

Whenever any of these events happen, the student can choose among a set of options, deciding on what subject he will work. Since some subjects

might be unknown to the student, LOCAL will suggest a subject to him, based on the data stored and processed by eProfile.

Suggestions are based on previous decisions taken by similar students. The first step in suggesting the result to a student is to calculate the similarity of all other students to him.

The level of similarity is defined by the number of the same previous choices (preferences) taken by two students. A set of the students most similar to a given student s is defined by M_s in

$$M_s = \left\{ x | x \in A \wedge \left(\frac{|P_s \cap P_x|}{|P_x|} > K \right) \right\}$$

where A is the set of all students (past and present), P_s is the set of the preferences of the student s, P_x is the set of preferences of the other student, and K is a parameter of the simulation that defines the level of similarity expected between students to suggest. Different tests were simulated for $K = \{0.3, 0.4, 0.5, 0.6, 0.7\}$.

The second step is to examine the choice made in each debate or paper by each student in M_s. The most chosen subject is the one suggested to the student.

Figure 12 shows the average results of the simulation, run 100 times for each K (percentage of similarity) in $\{0.3, 0.4, 0.5, 0.6, 0.7\}$. On this simulation, a group of 40 students goes through a complete undergraduation course. The course takes more than 16 semesters because, due to limited number of openings in the disciplines, not all students take all disciplines available every semester. The simulator registers the percentage of the debates and papers subjects that were suggested by LOCAL, based on the profile generated by eProfile that, in turn, was inferred from previous student choices.

An analysis of the results shows that the amount of successful recommendations improved over time in most percentages of similarity. This improvement happened because, as students were advancing in the course, more information about their preferences was being stored, thus building a more complete database. The amount of successful recommendations was also related to the reduction of the similarity required from other students. It is important to notice, however, that this might also result in a decrease in the quality of the suggestions. Due to the nature of the simulation, it is not possible to assess how much the quality of suggestions would change, but future experimentation with real users can be used the evaluate this information.

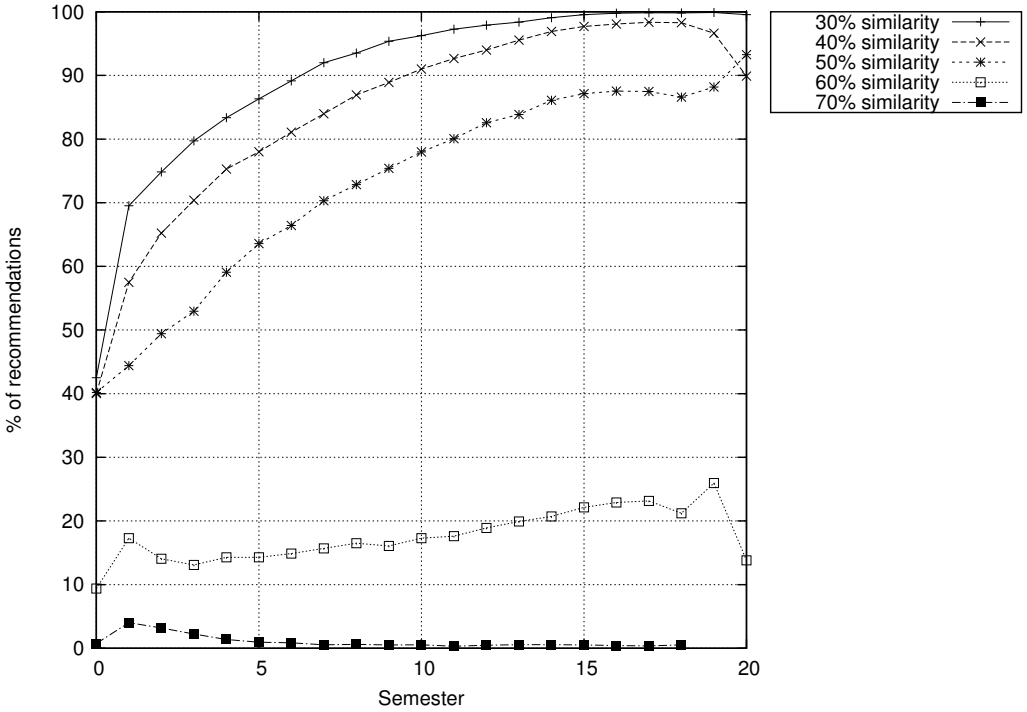

Figure 12: Simulation results: automatic recommendations by semester

One of eProfile main contributions is the possibility of changing the rules at runtime. To validate this functionality, the scenario described previously suffered a modification, in which a new rule was created.

This rule is based on the idea that if a student made most of the same choices as another student, their interests are similar, and the choices made by the second student may be used as recommendation for the first. The rule defines that, for each student, the 5 students who took most similar choices are chosen. Then, when a student was faced with multiple options, the previous choices made by these 5 students were used as recommendation.

A new simulation was made to test the capability of changing the rules during the execution of eProfile. In this test, a simulation was run using the same parameters as the previous one, with $K = 0.6$. In the 7th semester, it was found out that the percentage of recommendations was too low, and a new rule was registered stating that $K = 0.8$, and eProfile started to consider this rule (in addition to the previous rules) in the student profile inference. This was run for 5 semesters and, while the percentage of recommendations

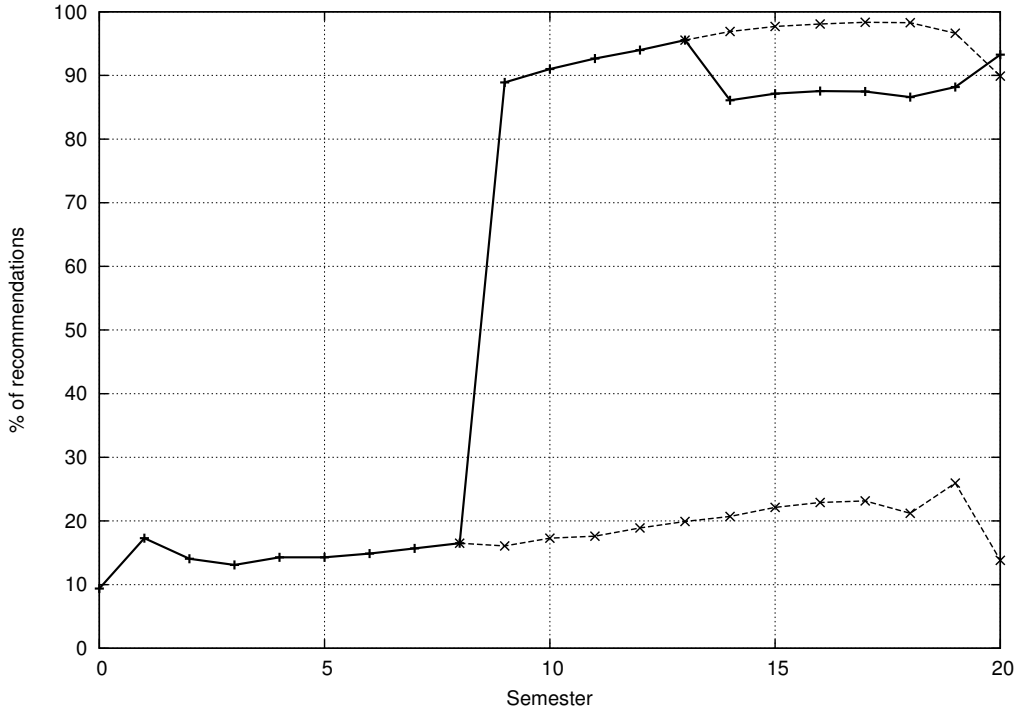

Figure 13: Simulation results with rule modification during execution

improved considerably, it was found out that the quality of the recommendations had fall ed too much. A new rule was then registered stating that $K = 0.7$ starting in the 13th semester.

Figure 13 shows the result of the second experiment. A sudden increase in the rate of recommended subjects can be seen from the 7th semester onward, and a decrease in the 13th semester, which is the result of the modification of the rules during the execution of the simulation. The dotted line shows how would the results continue if the new rules hadn't been applied.

6. Conclusion

This paper has proposed a model to support the management of profiles in interoperability environments, allowing entities trails inference through definition of rules by applications. Although eProfile was integrated with LOCAL and UniManager in the scenario, its proposal is generic enough to be applied to any other systems that use profiles from improvement of per-

sonalization. The use of automatic profiles allows systems to be more efficient in understanding user intent, because this means these systems can use more precise, up-to-date and broad information in a way that caters to the application specific need.

By updating the profiles with new information present in the trails, it is possible to keep the profile in sync with everything that is happening with the entity. This allows the application to personalize the user experience. In the scenario presented, the profiles are updated every time a student makes a decision regarding a debate or paper subject.

The semantic interoperability of the model opens the possibility that one application can use information generated by a different application to create the profile of its entities. This was demonstrated in the scenario where UniManager used information generated by LOCAL (the optional disciplines chosen by the students) to create the UniManager profiles. This information was later used to suggest optional disciplines to the students.

The scenario showed that is viable to integrate multiple systems with eProfile, enriching the profile generation of applications and allowing them to adapt themselves to their users.

The scenario demonstrated the main contributions of the model. The first contribution is the use of trails for extracting profiles. The use of trails enriches the information database that can be used for generating profiles by providing a historical view of the entity actions within contexts. In the scenario, past decisions regarding debate and papers subjects were used to extract the preferences that composed the students profiles.

A second contribution is the capability of managing dynamic inference rules and models. By allowing the rules and entity models to be updated at any time, eProfile facilitates the adaptation of the application to its user. A second experiment was made where the inference rules for profile generation were changed during the execution of the simulation, resulting in a positive change on the outcome of the scenario.

6.1. Future works

The eProfile is an initial proposal, such as its integration with the LOCAL and UniManager. According to the results obtained in the model validation, we present some proposals to continue the development and expansion of eProfile.

There is room to expand eProfile evaluations. eProfile features were validated through the simulation of a scenario. An ideal evaluation would involve

the generation of profiles by real entities where the model could be tested in a real environment, for a extended period of time. Although the simulation brings the advantages of multiple repetitions in a short time, an evaluation in a real environment would give more accurate results and would take into account aspects not covered in the simulation.

The simulation used a small set of entities, and no performance evaluation was made. Since the model is generic, other settings could use a larger number of entities and require profiles generated in a shorter period of time. Thus, the performance evaluation of eProfile implementation on a large number of trails in a short time could point to the need for improved performance.

The scenario showed eProfile integration of two applications (LOCAL and UniManager). Each application has its own characteristics, and eProfile integration with other applications may present opportunities for new functionality. In addition, integration with both applications occurred within an educational context. The integration with applications from other contexts may also indicate possibilities for implementation of new features.

As mentioned, the model is an initial proposal, and there are several opportunities for expanding. These include compatibility with other managers trails, expansion of basic ontologies, support to disconnected and/or unstable operation, server decentralization, and support for security protocols and confidentiality.

References

Abowd, G. D., Dey, A. K., Brown, P. J., Davies, N., Smith, M., Steggles, P., 1999. Towards a better understanding of context and context-awareness. In: Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing. HUC '99. Springer-Verlag, London, UK, pp. 304–307.

Augusto, J. C., Nakashima, H., Aghajan, H., 2010. Ambient intelligence and smart environments: A state of the art. In: Handbook of Ambient Intelligence and Smart Environments. pp. 3–31.

Barbosa, J. L. V., Hahn, R. M., Barbosa, D. N. F., da Costa Zanel Saccol, A. I., 2011. A ubiquitous learning model focused on learner interaction. International Journal of Learning Technology 2011 6 (1), 62 – 83.

Carmagnola, F., Cena, F., January 2009. User identification for cross-system personalisation. Information Sciences 179, 16–32.

Chellouche, S. A., Arnaud, J., Négru, D., 2010. Flexible user profile management for context-aware ubiquitous environments. In: Proceedings of the 7th IEEE conference on Consumer communications and networking conference. CCNC'10. IEEE Press, Piscataway, NJ, USA, pp. 980–984.

Daconta, M. C., Obrst, L. J., Smith, K. T., May 2003. The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management. Wiley, Indianapolis.

Dey, A., 2001. Understanding and using context. Personal and Ubiquitous Computing 6 (1), 4–7.

Dey, A. K., Abowd, G. D., Salber, D., Dec. 2001. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. Hum.-Comput. Interact. 16 (2), 97–166.

Driver, C., Clarke, S., October 2008. An application framework for mobile, context-aware trails. Pervasive Mob. Comput. 4, 719–736.

Edwards, W. K., Bellotti, V., Dey, A. K., Newman, M. W., 2003. The challenges of user-centered design and evaluation for infrastructure. In: Proceedings of the SIGCHI conference on Human factors in computing systems. CHI '03. ACM, New York, NY, USA, pp. 297–304.

Fielding, R. T., 2000. Architectural styles and the design of network-based software architectures. Ph.D. thesis, aAI9980887.

Fischer, G., 2001. User modeling in human–computer interaction. User modeling and user-adapted interaction 11 (1), 65–86.

Franco, L. K., Rosa, J. H., Barbosa, J. L., Costa, C. A., Yamin, A. C., 2011. Mucs: A model for ubiquitous commerce support. Electronic Commerce Research and Applications 10 (2), 237 – 246.

Geraci, A., 1991. IEEE Standard Computer Dictionary: Compilation of IEEE Standard Computer Glossaries. IEEE Press, Piscataway, NJ, USA.

Heckmann, D., November 2005. Ubiquitous user modeling. Ph.D. thesis, Department of Computer Science, Saarland University, Germany.

Hong, J., Suh, E.-H., Kim, J., Kim, S., May 2009. Context-aware system for proactive personalized service based on context history. Expert Syst. Appl. 36 (4), 7448–7457.

Hwaci, 2013. Sqlite home page. `http://www.sqlite.org/`. Accessed in 03/11/2013.

Hwang, G.-J., Tsai, C.-C., Yang, S. J. H., 2008. Criteria, strategies and research issues of context-aware ubiquitous learning. Educational Technology & Society 11 (2), 81–91.

Hwang, G.-J., Wu, T.-T., Chen, Y.-J., 2007. Ubiquitous computing technologies in education. IJDET 5 (4), 1–4.

Kim, K.-S., Lee, J.-D., 2008. Profile management framework based on web services for providing personalized services. In: Proceedings of the 2008 International Conference on Convergence and Hybrid Information Technology. ICHIT '08. IEEE Computer Society, Washington, DC, USA, pp. 501–508.

Levis, D., Barbosa, J. L. V., Pinto, S. C. C. S., Barbosa, D. N. F., 2008. Automatic improvement of the learner's profile in ubiquitous systems. Braz J Inform Educ 16 (1), 29–41.

Mulvenna, M. D., Anand, S. S., Büchner, A. G., August 2000. Personalization on the net using web mining: introduction. Commun. ACM 43, 122–125.

Niederée, C., Stewart, A., Mehta, B., Hemmje, M., 2004. A Multi-Dimensional, Unified User Model for Cross-System Personalization. In: E4PIA Workshop 2004.

Oliveira, R. R., Noguez, F. C., da Costa, C. A., Barbosa, J. L. V., Prado, M. P., 2013. Swtrack: An intelligent model for cargo tracking based on off-the-shelf mobile devices. Expert Syst. Appl. 40 (6), 2023–2031.

Razmerita, L., 2011. An ontology-based framework for modeling user behavior - a case study in knowledge management. IEEE Transactions on Systems, Man, and Cybernetics, Part A 41 (4), 772–783.

Rossum, G., 1995. Python reference manual. Tech. rep., Amsterdam, The Netherlands, The Netherlands.

Satyanarayanan, M., 2001. Pervasive computing: Vision and challenges. IEEE Personal Communications 8, 10–17.

Segatto, W., Herzer, E., Mazzotti, C. L., Bittencourt, J. R., Barbosa, J., 2008. Mobio threat: A mobile game based on the integration of wireless technologies. Computers in Entertainment 6 (3).

Silva, J., Rosa, J., Barbosa, J., Barbosa, D., Palazzo, L., 2010. Content distribution in trail-aware environments. Journal of the Brazilian Computer Society 16, 163–176.

Tao, D., Li, Y., Zhong, N., 2011. A personalized ontology model for web information gathering. IEEE Transactions on Knowledge & Data Engineering 23 (4), 496–511.

Tavares, J., Barbosa, J. L. V., da Costa, C. A., Yamin, A. C., Real, R. A., 2012. Hefestos: a model for ubiquitous accessibility support. In: Makedon, F. (Ed.), PETRA. ACM, p. 27.

Tsarkov, D., Horrocks, I., 2006. Fact++ description logic reasoner: System description. In: Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006). Vol. 4130 of Lecture Notes in Artificial Intelligence. Springer, pp. 292–297.

Viviani, M., Bennani, N., Egyed-Zsigmond, E., Aug. 2010. A Survey on User Modeling in Multi-Application Environments. In: The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10. IEEE, pp. 111–116.

W3C, 2011. OWL web ontology language guide. `http://www.w3.org/TR/owl-guide/`. Accessed in 03/11/2013.

Weiser, M., 1991. The computer for the 21st century. Scientific America, 94–104.

Weiser, M., Brown, J. S., July 1996. Designing calm technology. PowerGrid Journal.

Wu, P.-H., Hwang, G.-J., Su, L.-H., Huang, Y.-M., 2012. A context-aware mobile learning system for supporting cognitive apprenticeships in nursing skills training. Educational Technology & Society 15 (1), 223–236.